Lab 8: OpenTelemetry

Roll No: 12340100

Name: Aditya Yadav

Abstract: This report documents how I instrumented my Assignment 1 project, JalaSetu, with OpenTelemetry and ran it on the four-node cluster (sys1–sys4). Every analysis request now produces a distributed trace that follows the request across three machines, domain metrics for each pipeline stage, and structured logs that carry the trace ID of the request that wrote them. The data is exported over OTLP to an OpenTelemetry Collector and stored in Tempo (traces), Prometheus (metrics) and Loki (logs), and I inspect it in Grafana. All screenshots below are from my own running deployment on 28 October 2026.

1. Objective

The objective was to integrate OpenTelemetry into the existing Village Pond Planning System, capture traces, metrics and logs from the backend services, run the application, and show with an observability backend that the data is actually collected, exported and useful for monitoring and debugging.

2. The application being observed

JalaSetu takes a location (latitude, longitude, radius) and a DEM resolution, and returns candidate pond sites with their upstream catchment and estimated runoff. The pipeline is split across three backend nodes. Each node is a Flask (WSGI) app served by Uvicorn and managed by supervisord:

Node	Role	OpenTelemetry service.name
sys4 (172.17.0.5)	Gateway. Receives POST /analyzeContour through Nginx on port 3204, calls sys2 then sys3, then runs catchment delineation and runoff estimation itself	jalasetu-sys4-gateway
sys2 (172.17.0.3)	Parses the contour KML and builds the DEM (POST /v1/prepare)	jalasetu-sys2-dem
sys3 (172.17.0.4)	Terrain analysis and pond-site ranking (POST /v1/analyze)	jalasetu-sys3-terrain
sys1 (172.17.0.2)	Static frontend, plus the observability backend for this lab	—

Because one user request touches three machines, a log line or a timing on a single machine is not enough to understand a slow or failed request. That is the main reason distributed tracing is useful for this project.

3. OpenTelemetry implementation

3.1 SDK setup (app/telemetry.py)

A single function, init_telemetry(), is called once by each node at startup. It configures all three signals with OTLP/gRPC exporters pointed at the collector:

- Traces: TracerProvider with a BatchSpanProcessor and OTLPSpanExporter. Spans are buffered and sent in batches, so the request path never waits on the network.
- Metrics: MeterProvider with a PeriodicExportingMetricReader that pushes every 15 seconds.

- Logs: LoggerProvider with a BatchLogRecordProcessor. A LoggingHandler is attached to Python's root logger, so ordinary logger.warning(...) and logger.exception(...) calls become OpenTelemetry log records. Each record automatically carries the active trace_id and span_id.
- Resource: every signal is tagged with service.name (from OTEL_SERVICE_NAME), which is how sys2, sys3 and sys4 are told apart in every backend.
- Safety: a log filter strips filesystem paths and raw upload bytes before export. If the collector is unreachable, the app keeps serving requests; it only logs the export failure.

Configuration is only environment variables, set in each node's supervisord program entry:

```
OTEL_SERVICE_NAME=jalasetu-sys4-gateway        # sys2/sys3 use their own names
OTEL_EXPORTER_OTLP_ENDPOINT=http://172.17.0.2:4317
JALASETU_OTEL_ENABLED=true
```

3.2 Automatic instrumentation

- opentelemetry-instrumentation-flask creates a server span for every incoming HTTP request (route, method, status code).
- opentelemetry-instrumentation-requests creates a client span for every outgoing call and injects the W3C traceparent header. This is what joins the spans of sys4, sys2 and sys3 into a single trace: sys2 and sys3 read the header and continue the gateway's trace instead of starting new ones.

3.3 Manual instrumentation (domain spans and metrics)

Automatic spans only say that an HTTP call happened. To see inside the pipeline, I added a parent span analyze_contour on the gateway and one span per pipeline stage:

Span (stage → inner function span)	**Runs on**	**Attributes recorded (real values from Figure 5's trace)**
analyze_contour (parent)	sys4	dem_resolution_m=5, num_contours=2711, candidate_count=5, catchment_area_m2=2025, runoff_volume_m3=83.3, routing_method, has_upload
parse_contours → kml_parser.parse	sys2	num_contours, kml.num_features=2711, kml.source_type
build_dem → dem_builder.build_dem	sys2	dem.resolution_m=5, dem.width=650, dem.height=527, dem.crs=EPSG:32644, dem.min/max_elevation=30/298
terrain_analysis → terrain_analysis.analyze_terrain	sys3	terrain.mean_slope_deg≈3.74, terrain.mean_suitability≈56.6, terrain.suitable_area_percentage≈44.2
rank_pond_sites → pond_site.identify_candidate_sites	sys3	pond.candidates_count=5, pond.top_site_id, pond.top_suitability_score≈91.2, pond.top_storage_capacity_m3=450
delineate_catchment → catchment_delineation.delineate	sys4	catchment.area_m2, catchment.area_ha, catchment.method_used
estimate_runoff	sys4	(timing only)

Custom metrics (meter "jalasetu", app/metrics.py). Prometheus replaces the dots with underscores and adds a unit suffix:

Instrument	Type	Name in Prometheus	Purpose
jalasetu.analyses.total	Counter	jalasetu_analyses_total{status}	Throughput and error count
jalasetu.analysis.duration	Histogram (s)	jalasetu_analysis_duration_seconds_*	End-to-end latency, p50/p95
jalasetu.stage.duration	Histogram (s)	jalasetu_stage_duration_seconds_*{stage}	Time per pipeline stage
jalasetu.runoff.volume	Histogram (m^3)	jalasetu_runoff_volume_m3_*	Distribution of domain output
jalasetu.pysheds.fallback.total	Counter	jalasetu_pysheds_fallback_total	How often the fallback router is used

4. Tools, libraries and backend services

Component	Version	Where it runs	Purpose
opentelemetry-api / -sdk	1.41.1	sys2, sys3, sys4	Tracer, meter and logger providers
opentelemetry-exporter-otlp	1.41.1	sys2, sys3, sys4	OTLP/gRPC export of all three signals
instrumentation-flask / -requests / -logging	0.62b1	sys2, sys3, sys4	Automatic HTTP spans and context propagation
OpenTelemetry Collector Contrib	0.164.0	sys1 :4317 (gRPC), :4318 (HTTP)	Single OTLP entry point; batches and routes each signal
Grafana Tempo	3.1.1	sys1 :3200	Trace storage and TraceQL search
Prometheus	3.15.1	sys1 :9090	Scrapes the collector's metrics endpoint (:8889) every 10 s
Grafana Loki	3.7.9	sys1 :3100	Log storage (native OTLP ingestion)
Grafana OSS	13.3.0	My laptop, via an SSH tunnel to sys1	Visualisation: dashboard and Explore

Collector pipelines: traces go to Tempo over OTLP; metrics go to a Prometheus exporter on :8889 with resource_to_telemetry_conversion enabled, so service.name becomes a service_name label; logs go to Loki's OTLP endpoint. Each pipeline uses a memory_limiter and a batch processor.

4.1 Deployment constraints and decisions

sys1–sys4 are themselves unprivileged containers with a 512 MiB memory limit each, so Docker could not be used. Every backend runs as a plain native binary started by nohup scripts, with a GOMEMLIMIT cap per process. I first put Grafana on sys1 as well. Under load the node reached about 453 MiB of anonymous memory and started thrashing, and backends stalled. Grafana 13 alone used about 150 MiB plus one helper process per bundled datasource plugin. I therefore moved Grafana to my laptop, where it reads Tempo, Loki and Prometheus through an SSH tunnel. With only the collector and the three stores on sys1, the node peaked at about 275 MiB during the load test with zero memory pressure. The collector remains the single OTLP endpoint for the whole cluster.

A second problem appeared in Loki: the cluster's shared disk is about 93% full, and Loki's write-ahead log refuses all writes above 90% disk usage (the error message misleadingly says "Ingester is shutting down"). Raising ingester.wal.disk_full_threshold to 0.98 fixed it. Retention is kept short (Tempo 2 h, Loki 24 h, Prometheus 6 h) to protect the disk.

5. Running and verifying the implementation

scripts/otel_demo.py sends a fixed workload through the real Nginx entry point on sys4: five valid analyses at DEM resolutions 5, 10, 15, 5 and 10 m, one invalid request (radius 10,000 m, outside the allowed 10–2000 m), and one request outside the bundled map extent. Output of the run used for the screenshots:

```
$ BASE_URL=http://127.0.0.1:3204 python scripts/otel_demo.py
Target endpoint: http://127.0.0.1:3204/analyzeContour
[Success 1/5 (dem_resolution_m=5m)] status=200 duration=13.17s
[Success 2/5 (dem_resolution_m=10m)] status=200 duration=6.80s
[Success 3/5 (dem_resolution_m=15m)] status=200 duration=5.18s
[Success 4/5 (dem_resolution_m=5m)] status=200 duration=14.79s
[Success 5/5 (dem_resolution_m=10m)] status=200 duration=6.98s
[Invalid request (selected_radius_m=10000)] status=422 duration=0.02s
[Outside extent (lat=21.30, lon=81.40)] status=200 duration=7.04s
```

I then confirmed each signal directly through the backend APIs (not only in Grafana):

- Tempo returned 6 analyze_contour traces for the run. Each has 17 spans across all three services (sys4 → sys2 → sys3 → sys4), and all six stage spans are present.
- Prometheus lists all jalasetu_* series with service_name labels for the right node.
- Loki holds log streams for jalasetu-sys2-dem, jalasetu-sys3-terrain and jalasetu-sys4-gateway, each record carrying trace_id and span_id.
- The invalid request produced its own trace (HTTP 422) and an ERROR log with the same trace_id.

6. Screenshots and what they demonstrate

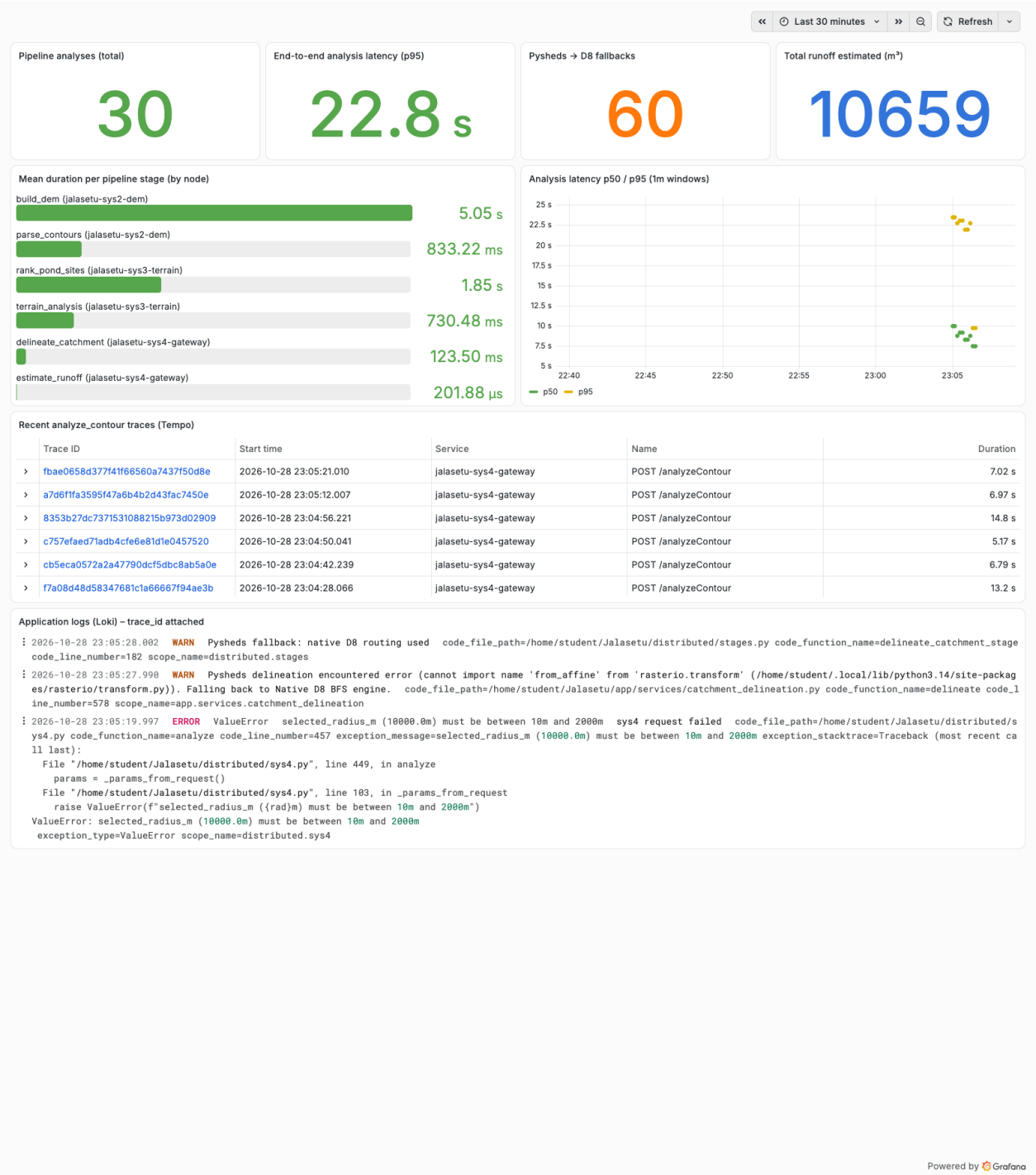


Figure 1: JalaSetu – OpenTelemetry Overview dashboard (Grafana, last 30 minutes)

What it shows: One dashboard that combines all three signals. Top row: total analyses, p95 end-to-end latency, number of Pysheds fallbacks, and total runoff estimated. Middle: mean time per pipeline stage, labelled with the node it ran on, and p50/p95 latency points for the load runs. Bottom: the recent analyze_contour traces from Tempo, and the application logs from Loki.

What I understand from it: This is the "is the system healthy?" view. It immediately shows that build_dem on sys2 (≈5.05 s mean) is by far the most expensive stage, more than all the others combined. It also shows that the Pysheds-fallback counter goes up on every analysis, which should never happen in normal operation. One lesson: the p95 tile says 22.8 s although the

slowest request took 14.8 s. histogram_quantile interpolates inside the SDK's default bucket (10–25 s), so a meaningful percentile needs explicit bucket boundaries around 5–20 s.

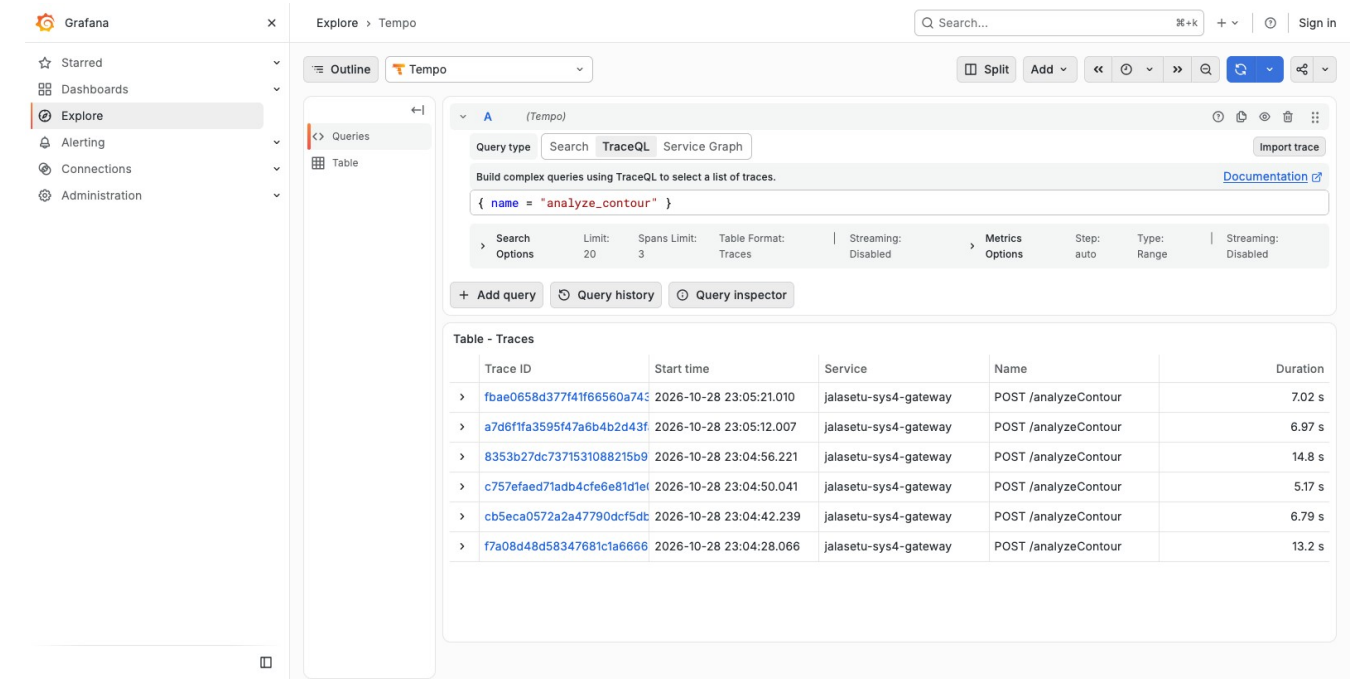


Figure 2: TraceQL search { name = "analyze_contour" } in Tempo

What it shows: Searching Tempo with TraceQL for every trace that contains an analyze_contour span. One row per demo request, with trace ID, start time, root service (the gateway) and duration.

What I understand from it: Traces are indexed and searchable by span name and attributes, not just by ID. The durations already group into two populations: about 13–15 s for the 5 m DEM requests and 5–7 s for the 10/15 m ones.

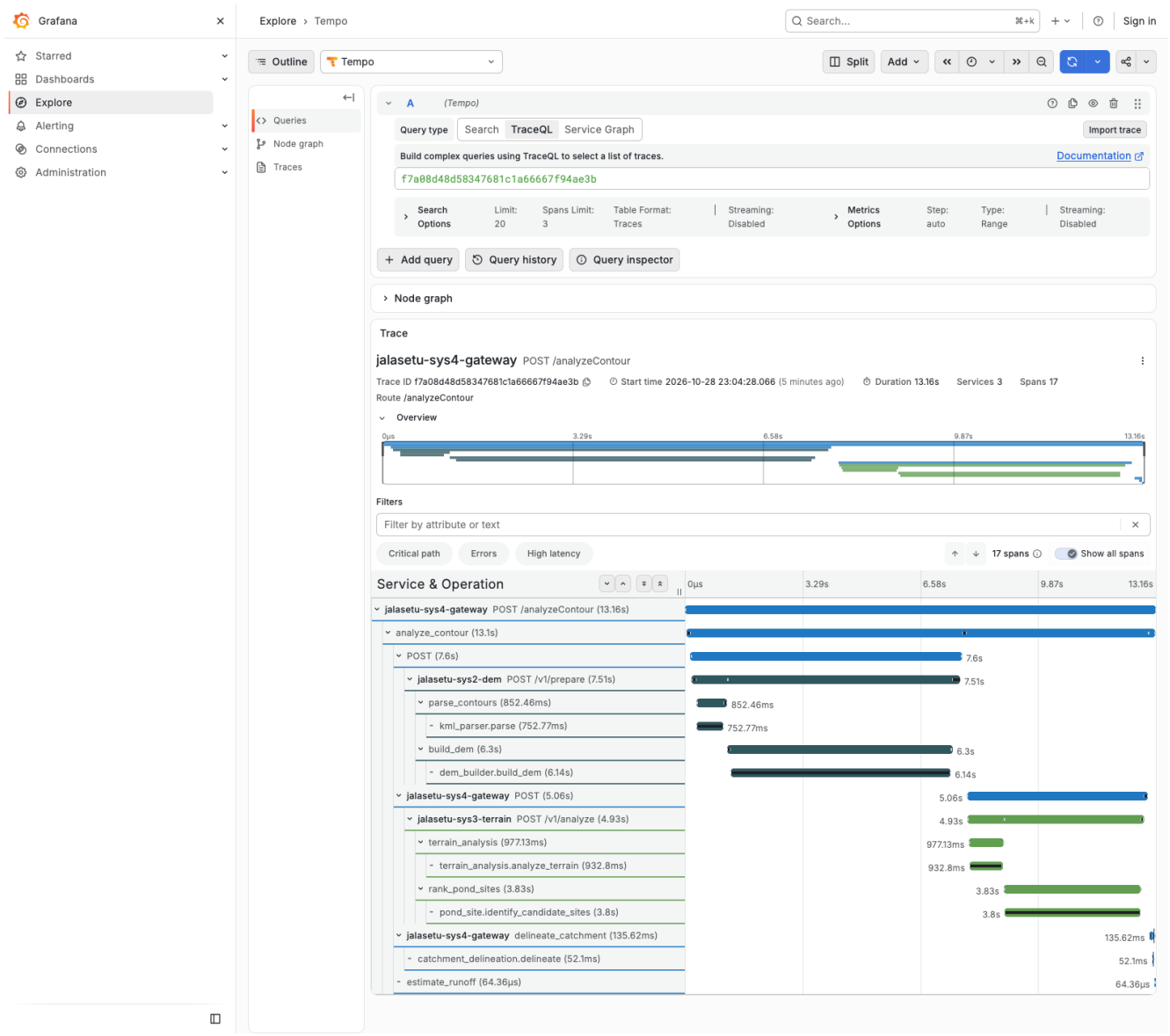


Figure 3: Distributed trace of one analysis (5 m DEM, 13.16 s, 17 spans, 3 services)

What it shows: The full waterfall of a single request. sys4 receives POST /analyzeContour and opens analyze_contour. It calls sys2 (POST /v1/prepare: parse_contours 852 ms, build_dem 6.3 s), then sys3 (POST /v1/analyze: terrain_analysis 977 ms, rank_pond_sites 3.83 s). Finally it runs delineate_catchment (136 ms) and estimate_runoff (64 µs) locally. Colours indicate the service.

What I understand from it: This is the main result of the assignment. One trace ID follows the request across three machines, because the requests instrumentation injected the traceparent header and Flask on sys2/sys3 continued the same trace. The waterfall shows that the work is strictly sequential: sys3 waits for sys2. For this request the critical path is build_dem (48% of the time) plus rank_pond_sites (29%). Optimising delineation or runoff would gain nothing measurable.

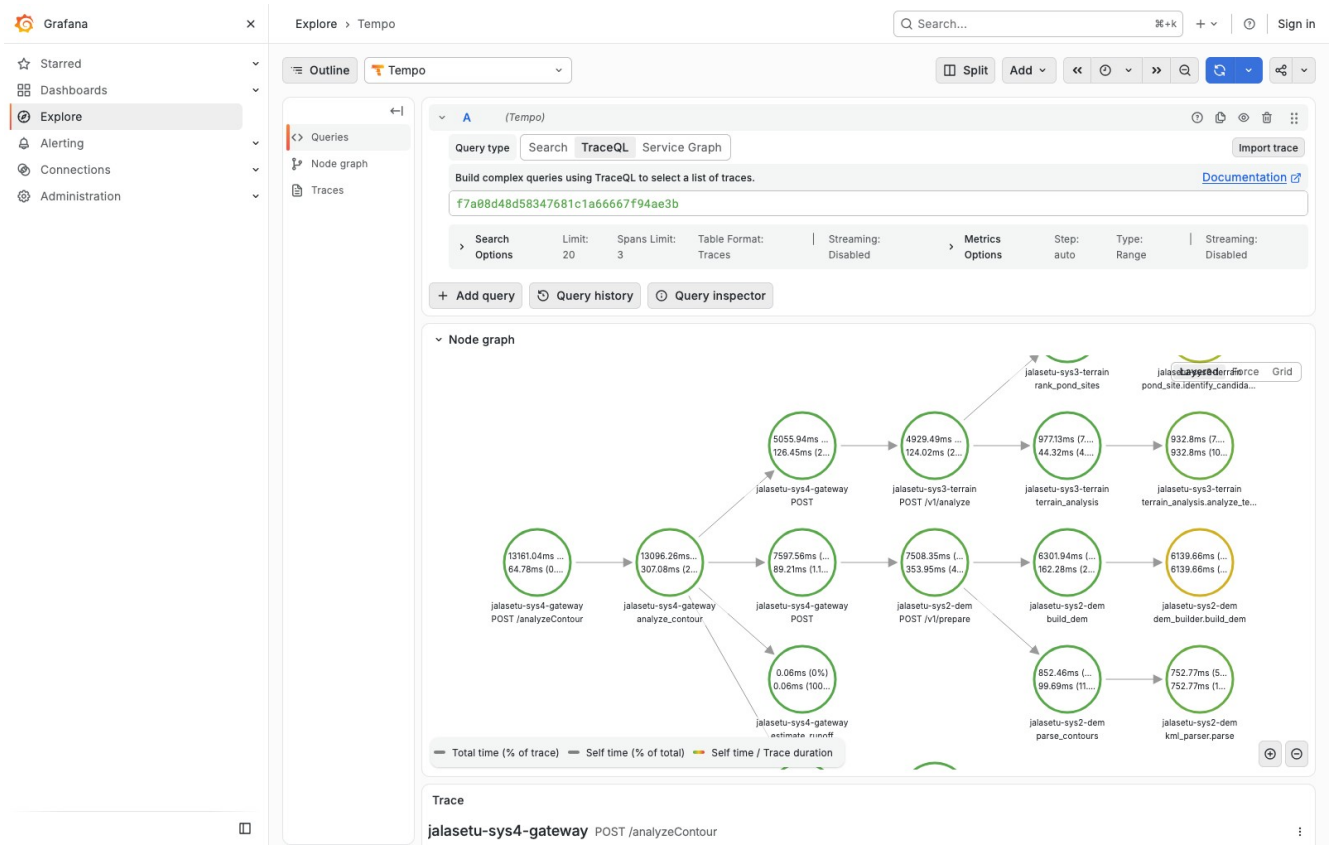


Figure 4: Node graph of the same trace

What it shows: Tempo's node graph view of the trace. Each circle is a service/operation pair, showing total time and self time, and the arrows are calls between them.

What I understand from it: This is a service map built from one request: gateway → DEM node → terrain node, and back. The yellow ring around dem_builder.build_dem marks a high self-time share, meaning the time is spent inside that function itself rather than in something it calls. That is where optimisation effort should go.

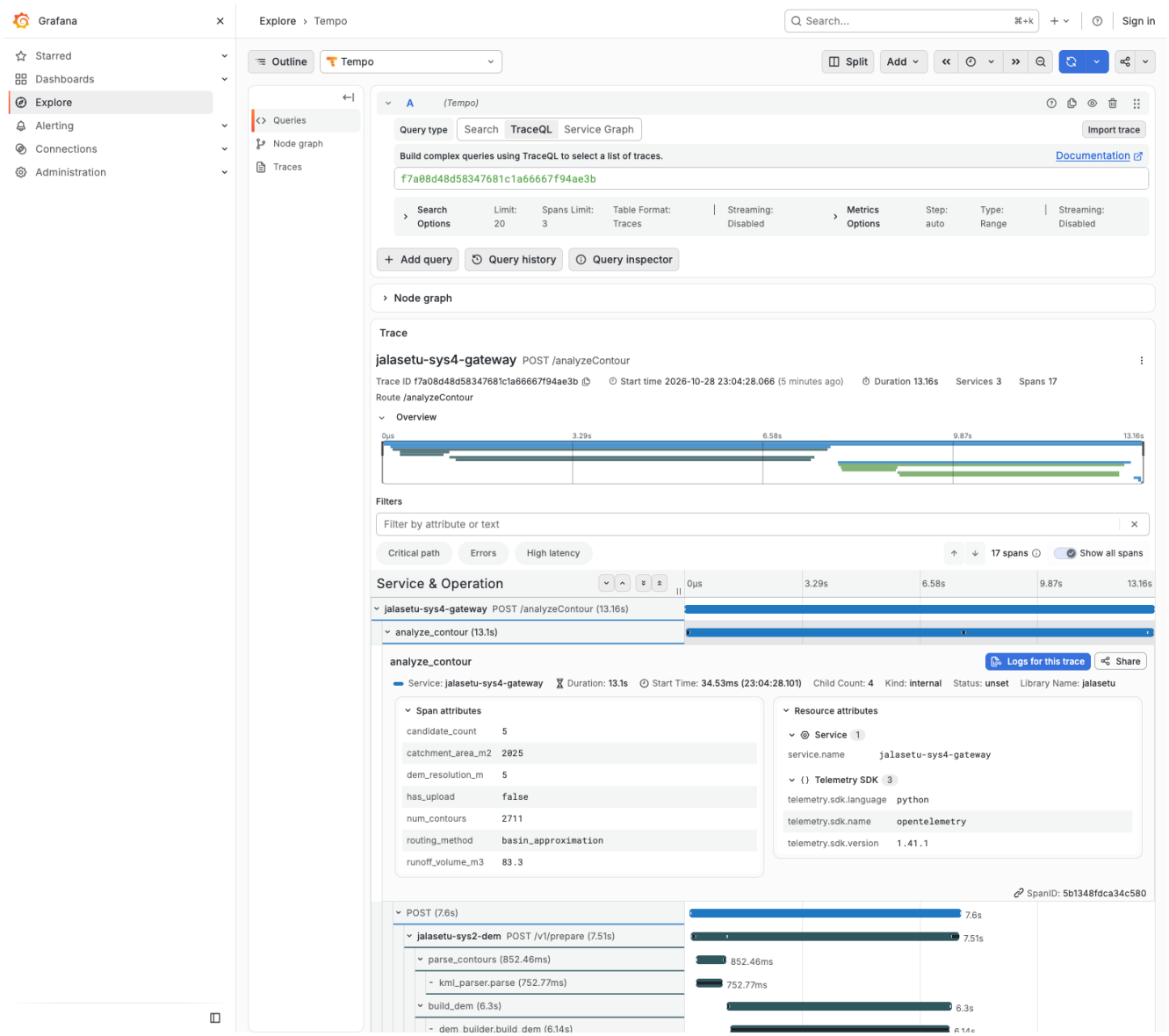


Figure 5: Span attributes and resource attributes of analyze_contour

What it shows: Clicking the analyze_contour span shows the domain attributes I record (dem_resolution_m=5, num_contours=2711, candidate_count=5, catchment_area_m2=2025, runoff_volume_m3=83.3, routing_method=basin_approximation). It also shows the resource attributes added by the SDK (service.name, telemetry.sdk.language=python, telemetry.sdk.version=1.41.1).

What I understand from it: Spans are not just timings. They carry the business context of the request, so a slow or wrong result can be explained from the trace alone, and comparing these attributes across traces shows how inputs change the output.

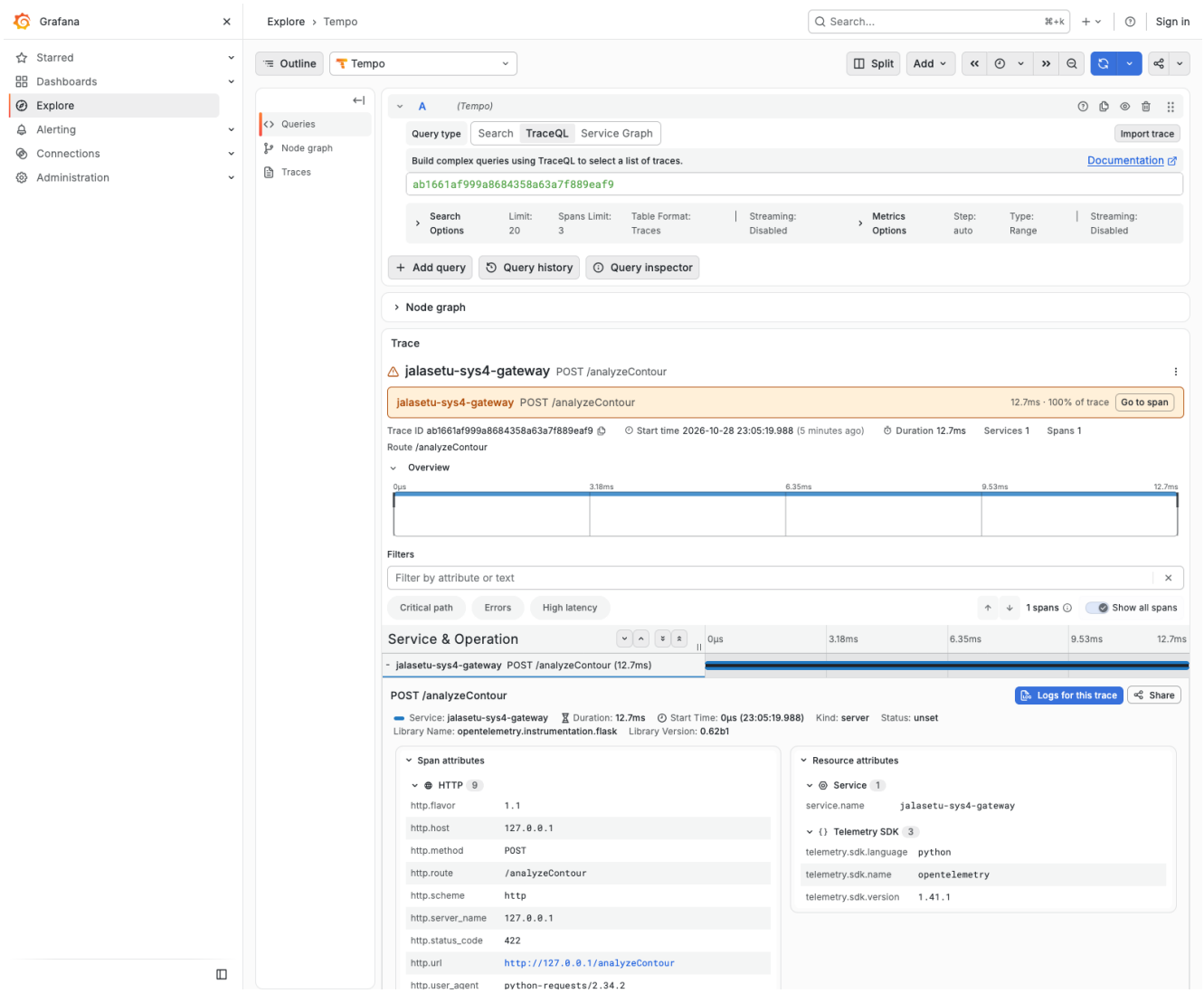


Figure 6: Trace of the invalid request (selected_radius_m = 10000) – HTTP 422

What it shows: The invalid request produced a single-span trace on the gateway (12.7 ms). Grafana flags it with a warning marker. The Flask-instrumentation span records http.status_code=422, http.route=/analyzeContour and the client user agent.

What I understand from it: Input validation rejected the request before any downstream call: there are no sys2 or sys3 spans, which proves bad input does not waste cluster time. The span status is "unset" rather than "error". That is correct per OpenTelemetry semantic conventions, which treat 4xx as a client error and not a server fault. Only 5xx would mark the server span as failed.

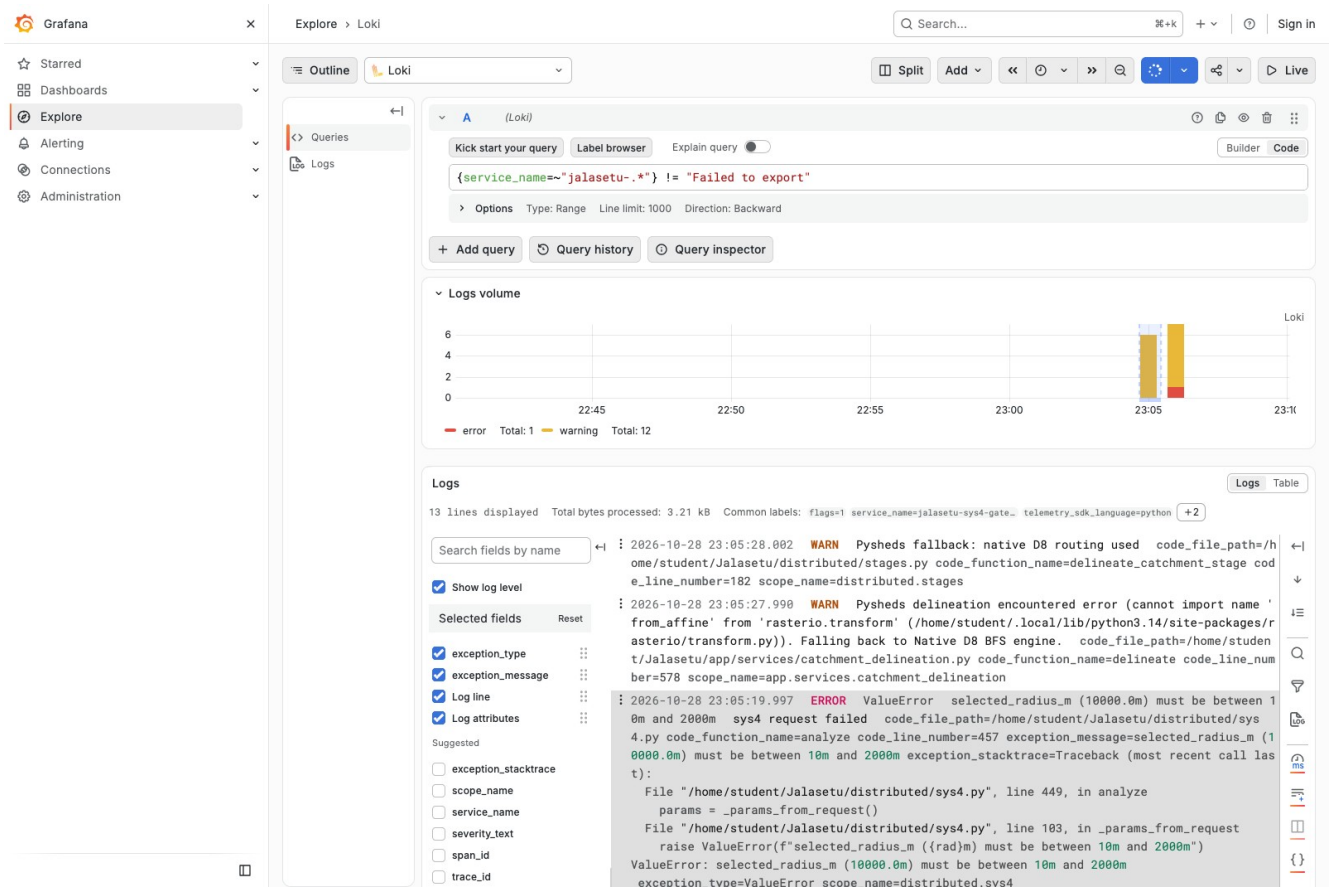


Figure 7: Application logs in Loki ({service_name=~"jalasetu-.*"})

What it shows: Log lines from the three services stored in Loki, with the log-volume histogram coloured by severity (warnings in yellow, the error in red). The structured fields (code file, function, line number, exception type and message) are kept as metadata, not just as text.

What I understand from it: The WARN lines show that Pysheds delineation fails with ImportError: cannot import name 'from_affine' from 'rasterio.transform', and falls back to the native D8 router. The ERROR line is the validation failure of the invalid request, including the full Python stack trace.

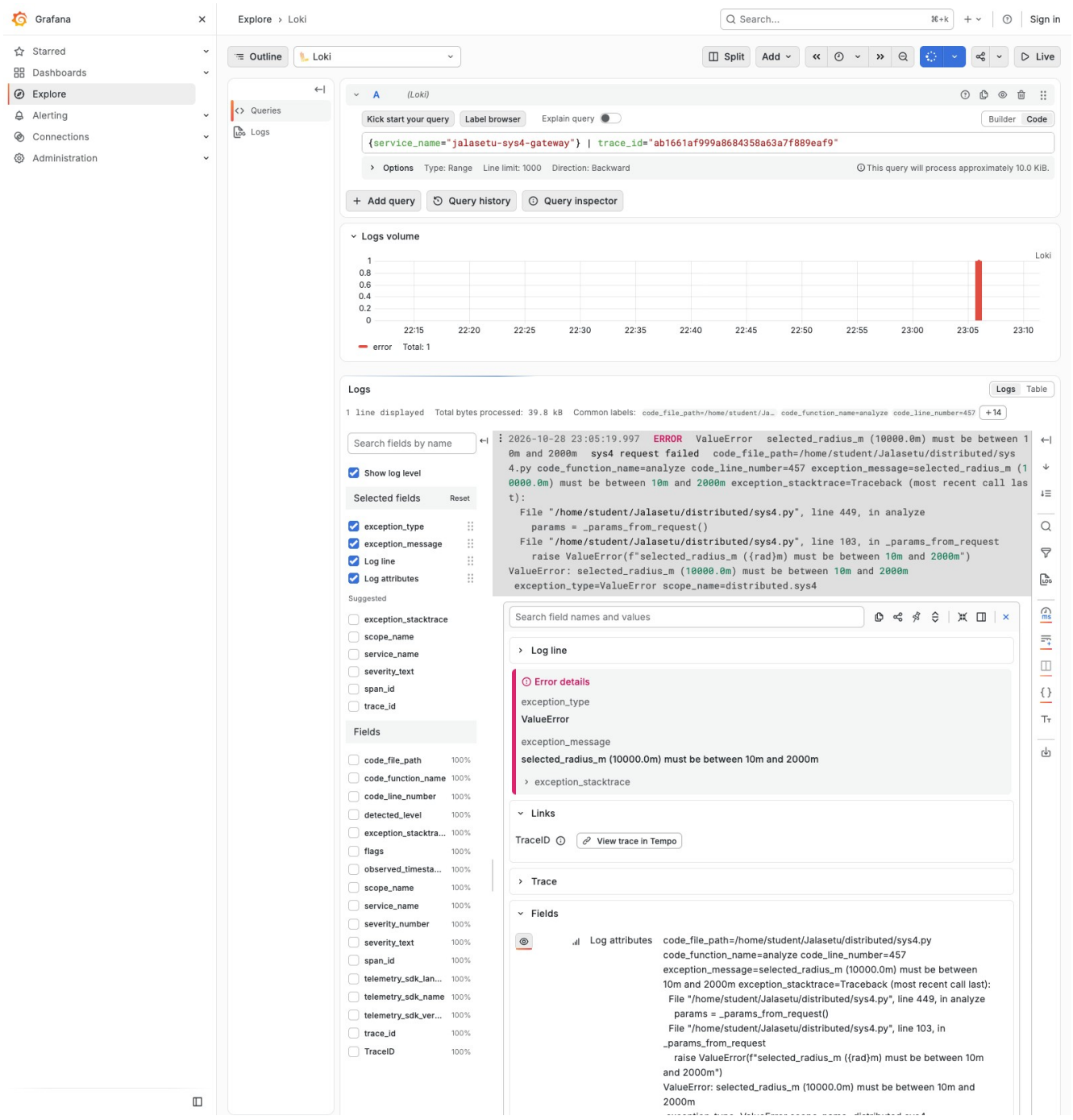


Figure 8: Log → trace correlation: the ERROR log of the invalid request

What it shows: Filtering Loki by the error trace's ID returns exactly one record: ERROR ValueError selected_radius_m (10000.0m) must be between 10m and 2000m, from distributed/sys4.py line 457. In the log details, the Links section shows the TraceID derived field with a "View trace in Tempo" button.

What I understand from it: Every log record carries the trace_id of the request that produced it. From an error log I can jump to the exact trace (Figure 6), and from a trace back to its logs ("Logs for this trace" button in Figures 5 and 6). This correlation is the practical difference between OpenTelemetry and plain log files: there is no need to search by timestamp across three machines.

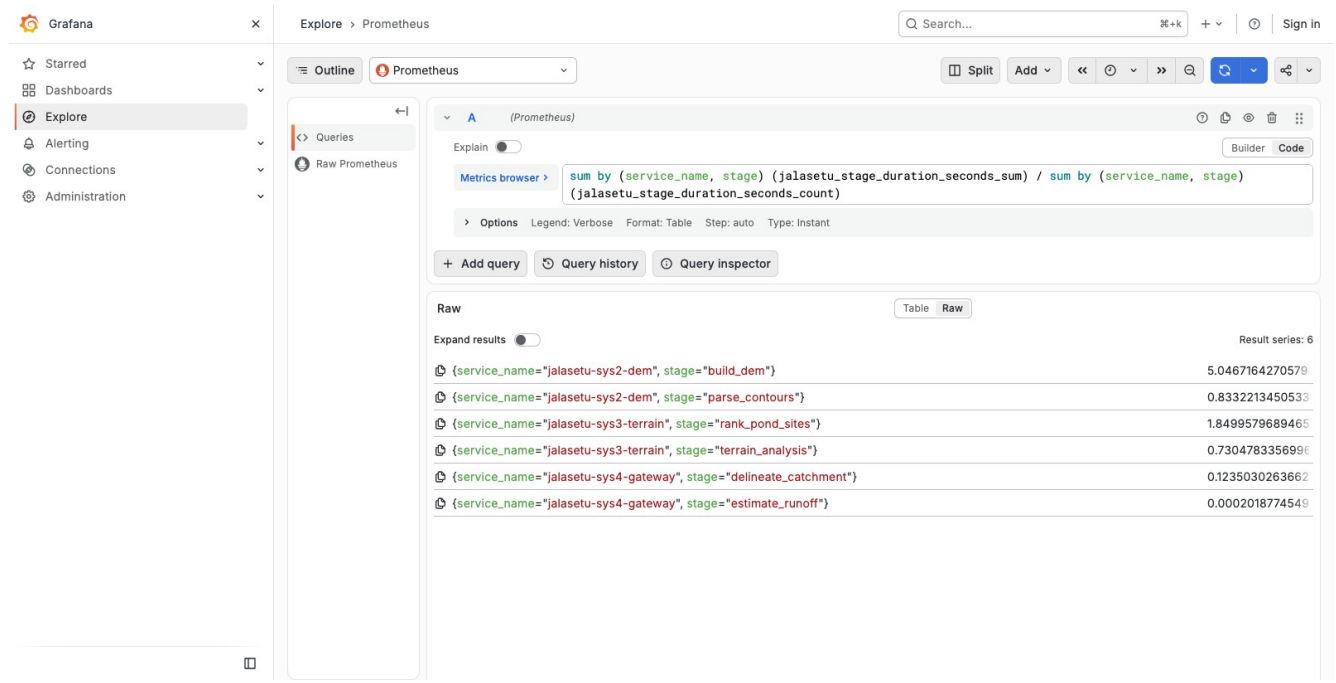


Figure 9: PromQL: mean stage duration per node

What it shows: Query: sum by (service_name, stage)(…_sum) / sum by (service_name, stage)(…_count). Result: build_dem 5.05 s, rank_pond_sites 1.85 s, parse_contours 0.83 s, terrain_analysis 0.73 s, delineate_catchment 0.12 s, estimate_runoff 0.2 ms.

What I understand from it: Traces explain one request, and metrics summarise all of them cheaply. The averages agree with the single trace in Figure 3, which shows the two signals are consistent. Metrics are what I would alert on, for example when the p95 of build_dem goes above a threshold.

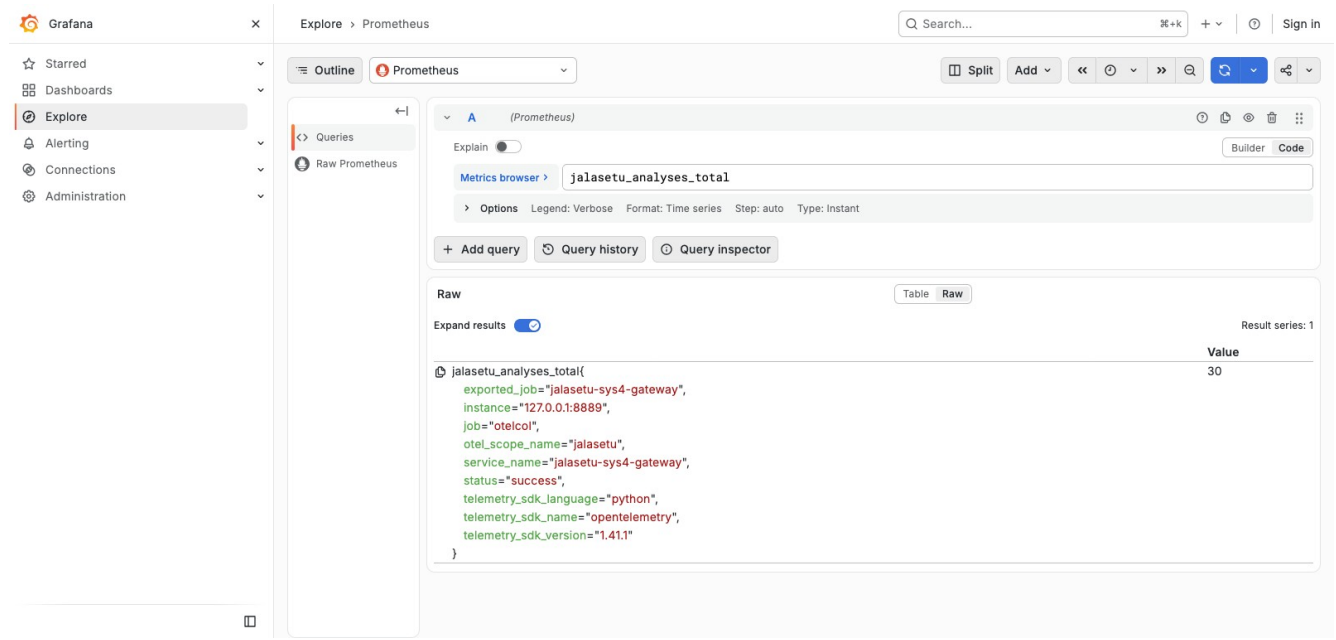


Figure 10: Raw metric series with resource attributes as labels

What it shows: The jalasetu_analyses_total series with all labels expanded: service_name=jalasetu-sys4-gateway, status=success, otel_scope_name=jalasetu, telemetry_sdk_* and job=otelcol. Value: 30 successful analyses.

What I understand from it: This shows the export path end to end. The SDK on sys4 pushed the counter over OTLP, the collector converted resource attributes into Prometheus labels (resource_to_telemetry_conversion), and Prometheus scraped it from the collector. The invalid request is not counted here because it was rejected before the pipeline started.

7. My understanding: how OpenTelemetry helps monitor, debug and analyse JalaSetu

7.1 Three signals, three different questions

- Metrics answer "is something wrong, and how much?" They are cheap aggregates: throughput, latency percentiles, per-stage averages and the fallback counter. They suit dashboards and alerts.
- Traces answer "where in this request did the time or the failure go?" One trace shows the causal path across sys4, sys2 and sys3 with timings and attributes for each step.
- Logs answer "what exactly happened?" They hold the exception message and stack trace. Because each log carries trace_id, I always know which request a log line belongs to.

Because all three share the same resource (service.name) and the same context (trace_id/span_id), I can move between them: from a metric spike, to slow traces in that time window, to the logs of one of those traces.

7.2 Design points worth noting

- Batching and asynchronous export keep telemetry off the request's critical path. A collector outage degrades observability only; requests keep working.
- Context propagation (W3C traceparent) is what makes a distributed trace possible. Without it, sys2 and sys3 would each produce isolated, unconnected traces.
- The collector decouples the app from the backends: the apps only know one OTLP endpoint, and I changed storage and visualisation placement without touching application code.
- OTLP is vendor neutral. The same instrumentation could export to Jaeger, Elastic or a hosted backend by changing collector configuration only.

8. Known limitations

- Retention is deliberately short (Tempo 2 h) because of the cluster's disk and memory limits. The data in the screenshots is not kept long term.
- Grafana runs on my laptop and reaches the cluster through an SSH tunnel, so the dashboard is only available while the tunnel is open.
- All spans are sampled (no sampling), which is fine at this request rate but would need a ratio or tail-based sampler at higher traffic.
- The 422 validation failure is recorded as a client error, not counted in jalasetu_analyses_total{status="error"}, because it is rejected before the pipeline starts.

9. Conclusion

OpenTelemetry is integrated into all three JalaSetu backend services and exports traces, metrics and logs through a single collector. A request entering sys4 can be followed through sys2 and sys3 in one trace. Stage-level metrics show where the time is spent, and every log line links back to its trace. Together these make the distributed system observable instead of a set of separate black boxes.